

Coding & Solution

Date	29 June 2026
Team ID	
Project Name	Rising water
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/gireesh2658/Rising_water
Programming Language(s)	Python (Backend & ML), HTML5, CSS3 (Frontend)
Framework(s) Used	Flask (Web Framework), XGBoost (ML Framework), Scikit-Learn (Data Processing), SQLAlchemy (Database ORM)
Key Features Implemented	1. Secure User Authentication (Login/Register with password hashing). 2. Real-time Flood Probability Prediction using trained XGBoost model. 3. Persistent Prediction History stored in PostgreSQL. 4. High-performance /health API for system monitoring. 5. Flask-Caching implemented for rapid history retrieval.
Pending / Incomplete Features	None. All core functional requirements, including ML inference and database integration, have been successfully implemented and tested.
Setup / Run Instructions	1. Install dependencies: <code>pip install -r requirements.txt</code> 2. Set environment variables (e.g., <code>FLASK_APP=app.py</code>, <code>DATABASE_URL</code>). 3. Initialize database (if applicable). 4. Run application: <code>python app.py</code> 5. Access via browser at <code>http://127.0.0.1:5000</code>

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The provided solution effectively bridges the gap between advanced Machine Learning and user-friendly web interfaces. By leveraging XGBoost for rapid inference and Flask for lightweight routing, the application strictly adheres to the performance and scalability requirements defined in the design phase. The codebase is fully PEP-8 compliant and utilizes standard MVC architectural patterns to ensure long-term maintainability.